Employee Management System Fullstack Project

🚀 Project Requirements & Architecture

- CRUD REST APIs for Employee entity: add, get by ID, get all, update, delete.
- Separate backend Spring Boot project exposing REST APIs.
- Separate frontend React project consuming backend APIs.
- Loosely coupled client-server model allowing frontend frameworks interchangeability (React, Angular, Vue).
- Backend three-layer architecture: Controller (REST endpoints), Service (business logic), DAO/Repository (DB interaction).
- Frontend organized with Components, Services, and Router.
- Use of JSON serialization/deserialization for API data exchange.

🖥️ Frontend Implementation Details

React Project Setup

- Created using Vite with JavaScript template.
- Bootstrap installed via npm and imported for styling.
- React Developer Tools extension recommended for debugging.
- Project structure overview: package.json (dependencies/scripts), vite.config.js (dev server config), src folder (JSX source files).

Core Components

- ListEmployeeComponent: displays employees in a Bootstrap-styled table with columns for ID, first name, last name, email, and action buttons.
- EmployeeComponent: used for both adding and updating employees with form inputs for first name, last name, and email.
- HeaderComponent & FooterComponent: static header and footer with Bootstrap navbar and custom CSS for footer.

Routing

- React Router v6 used for client-side routing.
- Routes configured in App.jsx:"/" and "/employees" → ListEmployeeComponent"/add-employee" → EmployeeComponent (for adding)"/edit-employee/:id" → EmployeeComponent (for updating)
  - "/" and "/employees" → ListEmployeeComponent
  - "/add-employee" → EmployeeComponent (for adding)
  - "/edit-employee/:id" → EmployeeComponent (for updating)
- BrowserRouter wraps the application for routing context.

State Management & API Integration

- Axios used for HTTP requests.
- useState hook manages form inputs and employees list.
- useEffect hook fetches employee data on component mount or ID change.
- CRUD operations call backend APIs via EmployeeService.js methods.
- After add/update/delete operations, navigate to employees list and refresh data.
- Form validation implemented with stateful error messages and Bootstrap validation classes.

UI Features

- Bootstrap forms with validation feedback for required fields.
- Buttons for Add, Update, Delete actions with proper event handlers.
- Dynamic page titles based on action (Add Employee vs Update Employee).
- Confirmation and error handling included in console logs.

🛠️ Technology Stack

- Backend: Spring Boot 3, Spring Data JPA (Hibernate 6), MySQL database.
- Frontend: ReactJS 18+, Vite as build tool, Bootstrap CSS framework.
- Development tools: IntelliJ IDEA for backend, Visual Studio Code for frontend.
- Build tools and package managers: Maven for backend, npm for frontend.
- API testing: Postman client.
- HTTP client in React: Axios library.
- JSON data exchange via DTO (Data Transfer Object).

⚙️ Backend Implementation Details

Spring Boot Project Setup

- Created via Spring Initializr with dependencies: spring-boot-starter-web, spring-boot-starter-data-jpa, MySQL JDBC driver, Lombok.
- Java 17 used with Spring Boot 3.1.0 stable.
- Application properties configured for MySQL connection, Hibernate dialect, and auto DDL update.
- Project imported into IntelliJ IDEA.

Employee Entity & Repository

- Employee JPA entity with fields: ID (auto-generated), firstName, lastName, email (unique, not null).
- Repository interface extends JpaRepository<Employee, Long>, inherits CRUD methods.
- Repository implementation managed by Spring Data JPA, transactional by default.

Service & Controller Layers

- EmployeeService interface with methods for CRUD operations.
- EmployeeServiceImpl implements business logic, uses repository and mapper.
- EmployeeController exposes REST APIs with Spring MVC annotations (@RestController, @RequestMapping).
- Exception handling via custom ResourceNotFoundException with HTTP 404 response.

DTO & Mapper

- EmployeeDTO used to transfer data between client and server.
- EmployeeMapper converts between Employee JPA entity and EmployeeDTO.
- Reduces boilerplate in service and controller layers.

REST APIs Implemented

- Add employee (POST /api/employees)
- Get employee by ID (GET /api/employees/{id})
- Get all employees (GET /api/employees)
- Update employee (PUT /api/employees/{id})
- Delete employee (DELETE /api/employees/{id})

🔑 Key Insights Summary

[NoteGPT] AI Mind Map Generator